

LC 556: Next Greater Element III

Authored by: Arpit Raj, LNMIIT Jaipur | Date: September 20, 2026

DE Shaw

Goldman Sachs

Zoho

Problem Summary:

Basically for a given number we need to rearrange its digit and output a number greater than the original number which is *just greater* than original (i.e. Next greater).

Condition: If no such exist or answer doesn't fit in 32 bit integer → Return -1

Approach & Visualization

Example 1: 1 2 4 5 3

→ Approach: starts from back of number. Try to look for a greater number on right.

1	2	4	5	3
---	---	---	---	---

Here 5 is greater than 4. Swap them.

→

1	2	5	4	3
---	---	---	---	---

Result: 1 2 5 4 3

A case to consider (Example 2): 7 3 8 4

For next greater need to traverse from back.

7	3	8	4
---	---	---	---

Both 4 and 8 > 3, but we will choose 4 as it's the *next greater*. Hence just greater than 3 we will select 4.

Swap:

7	4	8	3
---	---	---	---

 (7483 and not 7834)

Crucial Observation: Even just a single swap isn't enough. 7483 isn't right as 7438 is even smaller. So we need some sort of *sorting* after swapping the remaining right part.

C++ Code Implementation

```
class Solution {
public:
    int nextGreaterElement(int num) {
        vector<int> arr;

        while(num>0){
            int dig=num%10;
            arr.push_back(dig);
            num=num/10;
        }

        reverse(arr.begin(),arr.end());

        //eg 7384
        //[7,3,8,4]

        //for next greater need to traverse from back
        //eg 3,8,9 next great is 3,9,8

        //hence just greater than 3 we will select 4
        //7483 and not 7834
        //even just a single swap isnt enough
        //7483 isnt right as 7438 is even smaller
        //so we need some sort of sorting after swapping

        int n=arr.size();
        bool flag=false;
        for(int i=n-2;i>=0;i--){
            int miner=10;
            int mineridx=-1;
            for(int j=i+1;j<n;j++){
                if(arr[j]>arr[i] && arr[j]<=miner){
                    miner=arr[j];
                    mineridx=j;
                }
            }

            if(mineridx!=-1){
                swap(arr[i],arr[mineridx]);
                sort(arr.begin()+i+1,arr.end());
                flag=true;
                break;
            }
        }

        if(!flag){
```